

SYDE 556/750

Simulating Neurobiological Systems
Lecture 3: Population Representation

Chris Eliasmith

September 17 & 22, 2026

- ▶ Slide design: Andreas Stöckel
- ▶ Content: Terry Stewart, Andreas Stöckel, Chris Eliasmith



UNIVERSITY OF
WATERLOO

FACULTY OF
ENGINEERING



Visual Cortex

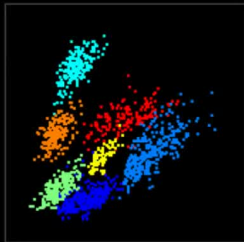


Mapping receptive fields

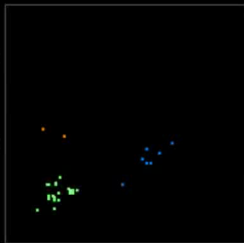
cell activity

behavior

overall



ongoing



NEF Principle 1: Representation

NEF Principle 1 – Representation

Groups (“populations”, or “ensembles”) of neurons *represent* values via nonlinear encoding and linear decoding.

Lossless Codes

INTERNATIONAL ALPHABET FLAGS, PHONETIC ALPHABET, MORSE CODE AND SEMAPHORE ALPHABET																															
A ALFA  Alphabet: 1st letter	B BRAVO  Alphabet: 2nd letter	C CHARLIE  Alphabet: 3rd letter	D DELTA  Alphabet: 4th letter	E ECHO  Alphabet: 5th letter	F FOXTROT  Alphabet: 6th letter	G GOLF  Alphabet: 7th letter	H HOTEL  Alphabet: 8th letter	I INDIA  Alphabet: 9th letter	J JULIETT  Alphabet: 10th letter	K KILO  Alphabet: 11th letter	L LIMA  Alphabet: 12th letter	M MIKE  Alphabet: 13th letter	N NOVEMBER  Alphabet: 14th letter	O OSCAR  Alphabet: 15th letter	P PAPA  Alphabet: 16th letter	Q QUEBEC  Alphabet: 17th letter	R ROMEO  Alphabet: 18th letter	S SIERRA  Alphabet: 19th letter	T TANGO  Alphabet: 20th letter	U UNIFORM  Alphabet: 21st letter	V VICTOR  Alphabet: 22nd letter	W WHISKEY  Alphabet: 23rd letter	X XRAY  Alphabet: 24th letter	Y YANKEE  Alphabet: 25th letter	Z ZULU  Alphabet: 26th letter	Alphabet  Alphabet: 27th letter	Hand  Hand: 28th letter	Error  Error: 29th letter	Repeater  Repeater: 30th letter	Answer  Answer: 31st letter	Doublet Sign Doublet Sign: 32nd letter
NAVAL NUMERAL FLAGS, PHONETIC NUMERALS AND MORSE CODE																															
1 ONE  Alphabet: 1st numeral	2 TWO  Alphabet: 2nd numeral	3 THREE  Alphabet: 3rd numeral	4 FOUR  Alphabet: 4th numeral	5 FIVE  Alphabet: 5th numeral	6 SIX  Alphabet: 6th numeral	7 SEVEN  Alphabet: 7th numeral	8 EIGHT  Alphabet: 8th numeral	9 NINE  Alphabet: 9th numeral	0 ZERO  Alphabet: 10th numeral																						
INTERNATIONAL NUMERAL PENNANTS																															
1 "Pennant One"  Alphabet: 1st pennant	2 "Pennant Two"  Alphabet: 2nd pennant	3 "Pennant Three"  Alphabet: 3rd pennant	4 "Pennant Four"  Alphabet: 4th pennant	5 "Pennant Five"  Alphabet: 5th pennant	6 "Pennant Six"  Alphabet: 6th pennant	7 "Pennant Seven"  Alphabet: 7th pennant	8 "Pennant Eight"  Alphabet: 8th pennant	9 "Pennant Nine"  Alphabet: 9th pennant	0 "Pennant Zero"  Alphabet: 10th pennant																						
NAVAL SPECIAL FLAGS AND PENNANTS																															
International Answer  Alphabet: 1st special flag	Code  Alphabet: 2nd special flag	Black Pennant  Alphabet: 3rd special flag	Corpus  Alphabet: 4th special flag	Designation  Alphabet: 5th special flag	Division  Alphabet: 6th special flag	Emergency  Alphabet: 7th special flag	Flotilla  Alphabet: 8th special flag	Formation  Alphabet: 9th special flag																							
Interrogative  Alphabet: 10th special flag	Negative  Alphabet: 11th special flag	Preparative  Alphabet: 12th special flag	Port  Alphabet: 13th special flag	Speed  Alphabet: 14th special flag	Squadron  Alphabet: 15th special flag	Starboard  Alphabet: 16th special flag	Station  Alphabet: 17th special flag	Submarine  Alphabet: 18th special flag	Tow  Alphabet: 19th special flag																						
First Substitute  Alphabet: 20th special flag	Second Substitute  Alphabet: 21st special flag	Third Substitute  Alphabet: 22nd special flag	Fourth Substitute  Alphabet: 23rd special flag																												

A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
P
Q
R
S
T

U
V
W
X
Y
Z

1
2
3
4
5
6
7
8
9
0

Encoding: $a = f(x)$

Decoding: $x = f^{-1}(a)$

Binary numbers: Nonlinear encoding, linear decoding

- Represent a natural number between 0 and $2^n - 1$ as n binary digits.

Binary numbers: Nonlinear encoding, linear decoding

- ▶ Represent a natural number between 0 and $2^n - 1$ as n binary digits.
- ▶ **Nonlinear encoding**

$$a_i = (f(x))_i = \begin{cases} 1 & \text{if } x - 2^i \lfloor \frac{x}{2^i} \rfloor \geq 2^{i-1}, \\ 0 & \text{otherwise.} \end{cases}$$

Binary numbers: Nonlinear encoding, linear decoding

- Represent a natural number between 0 and $2^n - 1$ as n binary digits.
- **Nonlinear encoding**

$$a_i = (f(x))_i = \begin{cases} 1 & \text{if } x - 2^i \lfloor \frac{x}{2^i} \rfloor \geq 2^{i-1}, \\ 0 & \text{otherwise.} \end{cases}$$

- **Linear decoding**

$$x = f^{-1}(\mathbf{a}) = \sum_{i=1}^n 2^{i-1} a_i = \mathbf{F}\mathbf{a} = \begin{pmatrix} 1 & 2 & \dots & 2^{n-1} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

Binary numbers: Nonlinear encoding, linear decoding

- Represent a natural number between 0 and $2^n - 1$ as n binary digits.
- **Nonlinear encoding**

$$a_i = (f(x))_i = \begin{cases} 1 & \text{if } x - 2^i \lfloor \frac{x}{2^i} \rfloor \geq 2^{i-1}, \\ 0 & \text{otherwise.} \end{cases}$$

- **Linear decoding**

$$x = f^{-1}(\mathbf{a}) = \sum_{i=1}^n 2^{i-1} a_i = \mathbf{F}\mathbf{a} = \begin{pmatrix} 1 & 2 & \dots & 2^{n-1} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

- This is a **distributed code**.

Binary numbers: Nonlinear encoding, linear decoding

- Represent a natural number between 0 and $2^n - 1$ as n binary digits.
- **Nonlinear encoding**

$$a_i = (f(x))_i = \begin{cases} 1 & \text{if } x - 2^i \lfloor \frac{x}{2^i} \rfloor \geq 2^{i-1}, \\ 0 & \text{otherwise.} \end{cases}$$

- **Linear decoding**

$$x = f^{-1}(\mathbf{a}) = \sum_{i=1}^n 2^{i-1} a_i = \mathbf{F}\mathbf{a} = \begin{pmatrix} 1 & 2 & \dots & 2^{n-1} \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

- This is a **distributed code**. But, **not robust** against additive noise!

Lossy codes

- **Lossy code**

Inverse f^{-1} does not exist, instead *approximate* the represented value

Encoding: $\mathbf{a} = f(\mathbf{x})$

Decoding: $\mathbf{x} \approx g(\mathbf{a})$

Lossy codes

- ▶ **Lossy code**

Inverse f^{-1} does not exist, instead *approximate* the represented value

Encoding: $\mathbf{a} = f(\mathbf{x})$

Decoding: $\mathbf{x} \approx g(\mathbf{a})$

- ▶ **Examples**

- ▶ Audio, image, and video coding schemes (MP3, JPEG, H.264)

- ▶ Basis transformation onto first n principal components (PCA)

Lossy codes

- ▶ **Lossy code**

Inverse f^{-1} does not exist, instead *approximate* the represented value

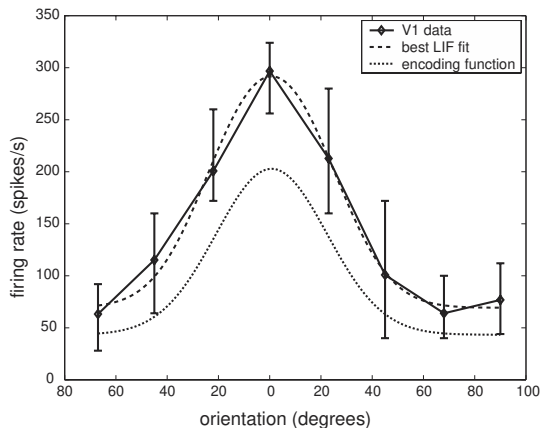
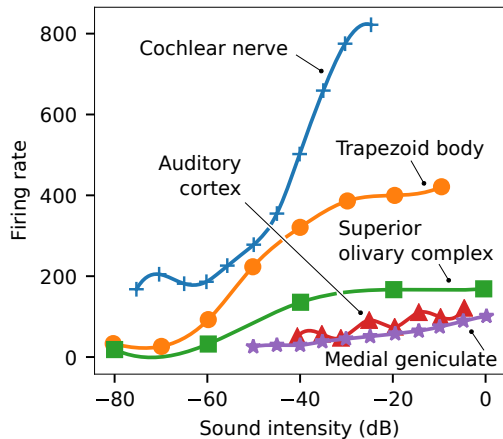
Encoding: $\mathbf{a} = f(\mathbf{x})$

Decoding: $\mathbf{x} \approx g(\mathbf{a})$

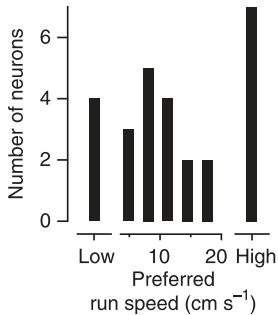
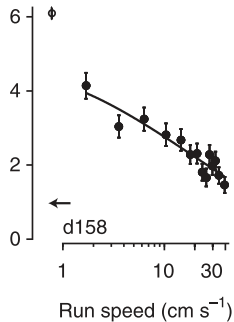
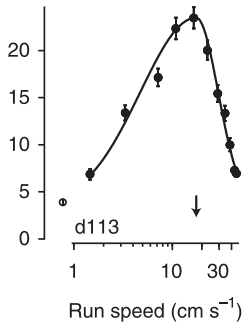
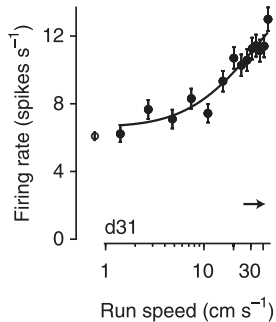
- ▶ **Examples**

- ▶ Audio, image, and video coding schemes (MP3, JPEG, H.264)
- ▶ Basis transformation onto first n principal components (PCA)
- ▶ **Neural Representations**

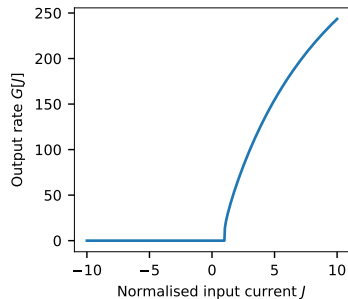
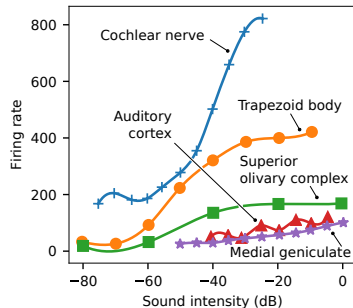
Tuning curves (I)



Tuning curves (II)



- Last lecture: response curves:
 $a = G(J)$
- This lecture: tuning curves:
 $a = f(x) = G(J_i(x))$
- What sort of function can we try for $J_i(x)$?

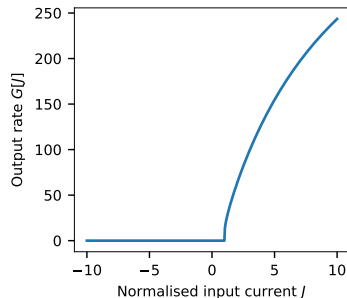
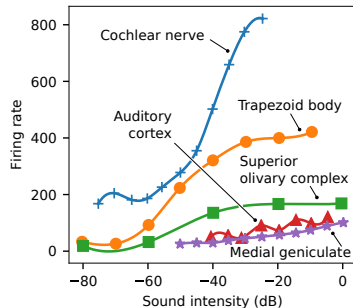


- Last lecture: response curves:
 $a = G(J)$
- This lecture: tuning curves:
 $a = f(x) = G(J_i(x))$
- What sort of function can we try for $J_i(x)$?
- Introduce a gain α_i and a bias J_i^{bias} :

$$J_i(x) = \alpha_i x + J_i^{\text{bias}}$$

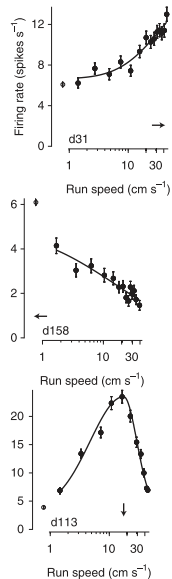
$$a_i(x) = G(\alpha_i x + J_i^{\text{bias}})$$

- α_i controls the slope
- J_i^{bias} shifts curve left and right



- Does this work for all tuning curves?

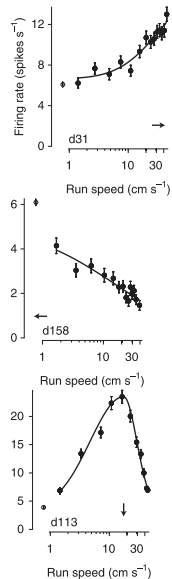
$$a_i(x) = G(\alpha_i x + J_i^{\text{bias}})$$



- Does this work for all tuning curves?

$$a_i(x) = G(\alpha_i x + J_i^{\text{bias}})$$

- a) increasing: Yes!

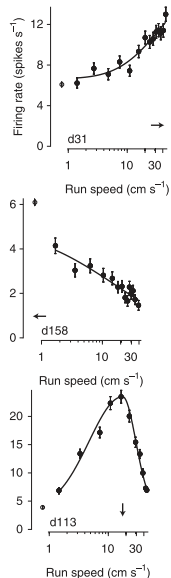


- Does this work for all tuning curves?

$$a_i(x) = G(\alpha_i x + J_i^{\text{bias}})$$

- a) increasing: Yes!
- b) decreasing: Yes! (just let α_i be negative)
 - or, better yet, introduce e_i which is either 1 or -1 and keep α_i to be always positive. This keeps the two ideas (slope and increase/decreasing) separate.

$$a_i(x) = G(\alpha_i(e_i x) + J_i^{\text{bias}})$$



- Does this work for all tuning curves?

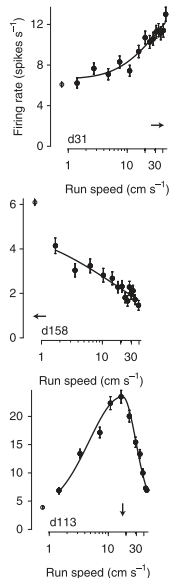
$$a_i(x) = G(\alpha_i x + J_i^{\text{bias}})$$

- a) increasing: Yes!
- b) decreasing: Yes! (just let α_i be negative)
 - or, better yet, introduce e_i which is either 1 or -1 and keep α_i to be always positive. This keeps the two ideas (slope and increase/decreasing) separate.

$$a_i(x) = G(\alpha_i(e_i x) + J_i^{\text{bias}})$$

- c) preferred stimulus: Need some sort of similarity measure
 - But it shouldn't be too complicated. So far we've only needed to introduce multiplication and addition, which are both things we're pretty sure neurons can do, so let's avoid adding anything else if we don't have to. Ideas?

$$a_i(x) = G(\alpha_i \text{sim}(e_i, x) + J_i^{\text{bias}})$$



Encoders: Preferred Direction Vectors

- ▶ The represented value x doesn't have to be a scalar
- ▶ What if it's a vector?

Encoders: Preferred Direction Vectors

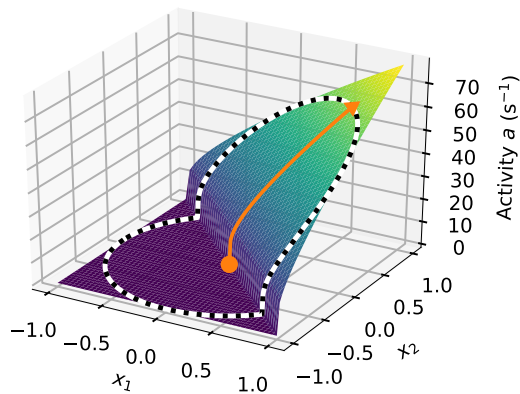
- ▶ The represented value x doesn't have to be a scalar
- ▶ What if it's a vector?
- ▶ There's a simple similarity-like measure for vectors: the dot product

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=0}^d x_i y_i = \cos(\angle(\mathbf{x}, \mathbf{y})) \|\mathbf{x}\| \|\mathbf{y}\|$$

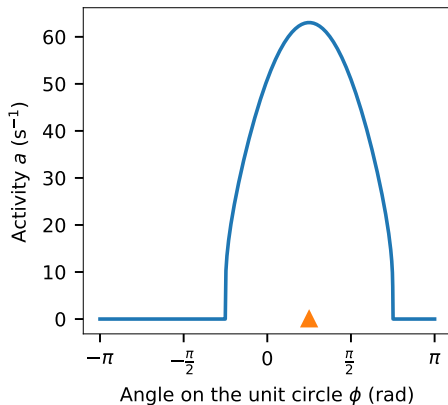
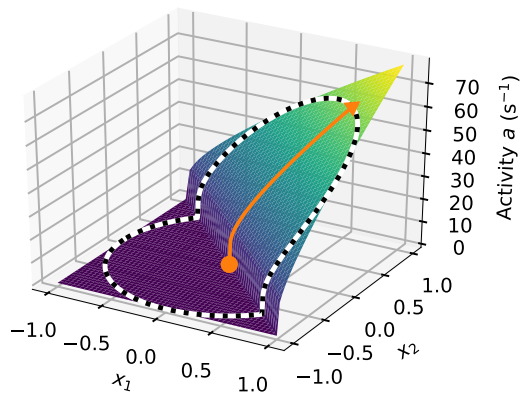
$$a_i(\mathbf{x}) = G(\alpha_i \langle \mathbf{e}_i, \mathbf{x} \rangle + J_i^{\text{bias}})$$

- ▶ Constrain $\mathbf{e}_i$ to be a unit vector
 - ▶ Note that for scalar x , the only two unit vectors are $+1$ and -1
 - ▶ So the increasing / decreasing scenario is a special case of this!

Preferred Directions in Higher Dimensions: Representing 2D Values



Preferred Directions in Higher Dimensions: Representing 2D Values



Decoding

- ▶ Non-linear Encoding and Linear Decoding

$$\mathbf{a}_i = G[\alpha_i \langle \mathbf{x}, \mathbf{e}_i \rangle + J_i^{\text{bias}}],$$

$$\hat{\mathbf{x}} = \mathbf{D}\mathbf{a}$$

Encoding

Decoding

- ▶ How do we find $\mathbf{D}$?

Decoding

- Non-linear Encoding and Linear Decoding

$$\mathbf{a}_i = G[\alpha_i \langle \mathbf{x}, \mathbf{e}_i \rangle + J_i^{\text{bias}}],$$

Encoding

$$\hat{\mathbf{x}} = \mathbf{D}\mathbf{a}$$

Decoding

- How do we find $\mathbf{D}$?
- Least-squares minimization

$$\arg \min_{\mathbf{D}} E = \frac{1}{|\mathbb{X}|} \int_{\mathbb{X}} \|\mathbf{x} - \hat{\mathbf{x}}\| \, d\mathbf{x} = \frac{1}{|\mathbb{X}|} \int_{\mathbb{X}} \|\mathbf{x} - \mathbf{D}\mathbf{a}(\mathbf{x})\| \, d\mathbf{x}$$

Decoding via Least-squares Minimization

- Find the minimum decoding error

$$\arg \min_{\mathbf{D}} E = \frac{1}{|\mathbb{X}|} \int_{\mathbb{X}} \|\mathbf{x} - \hat{\mathbf{x}}\| \, d\mathbf{x} = \frac{1}{|\mathbb{X}|} \int_{\mathbb{X}} \|\mathbf{x} - \mathbf{D}\mathbf{a}(\mathbf{x})\| \, d\mathbf{x}$$

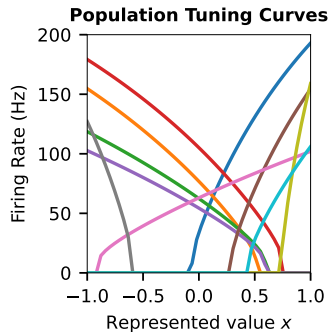
- We can do this analytically, or sample:

$$\arg \min_{\mathbf{D}} E = \frac{1}{N} \sum_{i=0}^N \|\mathbf{x}_i - \mathbf{D}\mathbf{a}(\mathbf{x}_i)\|$$

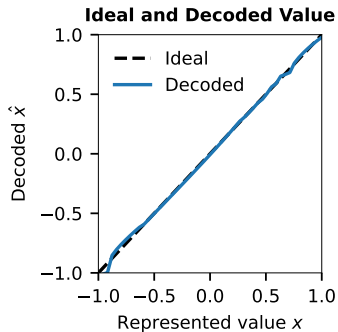
Decoding via Least-squares Minimization

- ▶ Let's write this in matrix form, where $\mathbf{A}_{ik} = a_i(x_k)$ and $\mathbf{X} = (x_1, \dots, x_N)$
- ▶ We want $\mathbf{A}^T \mathbf{D}^T = \mathbf{X}^T$
- ▶ So $\mathbf{A} \mathbf{A}^T \mathbf{D}^T = \mathbf{A} \mathbf{X}^T$
- ▶ $(\mathbf{A} \mathbf{A}^T)^{-1} \mathbf{A} \mathbf{A}^T \mathbf{D}^T = (\mathbf{A} \mathbf{A}^T)^{-1} \mathbf{A} \mathbf{X}^T$
- ▶ $\mathbf{D}^T = (\mathbf{A} \mathbf{A}^T)^{-1} \mathbf{A} \mathbf{X}^T$
- ▶ In Python, `D = np.linalg.lstsq(A.T, X.T, rcond=None)[0].T`
- ▶ (where $\mathbf{A}$ is a $n \times N$ array and $\mathbf{X}$ is a $d \times N$ array)

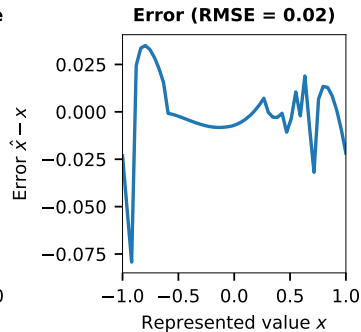
Decoding



A



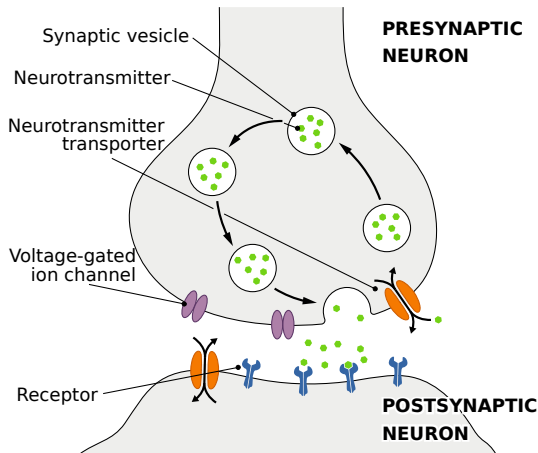
$\mathbf{A}^T \mathbf{D}^T$



$\mathbf{A}^T \mathbf{D}^T - \mathbf{X}^T$

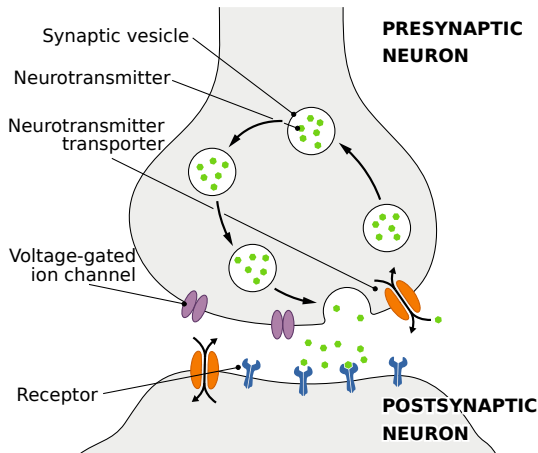
Sources of Noise in Biological Neural Networks

- ▶ **Axonal jitter**
Active axonal spike propagation
- ▶ **Vesicle release failure**
10-30% of pre-synaptic events cause post-synaptic current
- ▶ **Neurotransmitter per vesicle**
Varying amounts of neurotransmitter
- ▶ **Ion channel noise**
Ion-channels are “binary”, stochastic
- ▶ **Thermal noise**
- ▶ **Network effects**
Simple, noise-free inhibitory/excitatory networks produce irregular spike trains



Sources of Noise in Biological Neural Networks

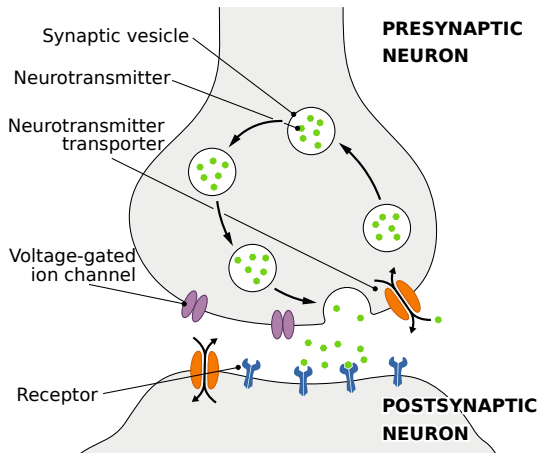
- ▶ **Axonal jitter**
Active axonal spike propagation
- ▶ **Vesicle release failure**
10-30% of pre-synaptic events cause post-synaptic current
- ▶ **Neurotransmitter per vesicle**
Varying amounts of neurotransmitter
- ▶ **Ion channel noise**
Ion-channels are “binary”, stochastic
- ▶ **Thermal noise**
- ▶ **Network effects**
Simple, noise-free inhibitory/excitatory networks produce irregular spike trains



▶ **How to model?**

Sources of Noise in Biological Neural Networks

- ▶ **Axonal jitter**
Active axonal spike propagation
- ▶ **Vesicle release failure**
10-30% of pre-synaptic events cause post-synaptic current
- ▶ **Neurotransmitter per vesicle**
Varying amounts of neurotransmitter
- ▶ **Ion channel noise**
Ion-channels are “binary”, stochastic
- ▶ **Thermal noise**
- ▶ **Network effects**
Simple, noise-free inhibitory/excitatory networks produce irregular spike trains



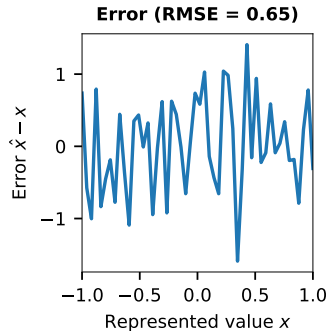
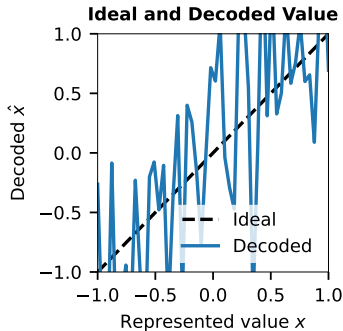
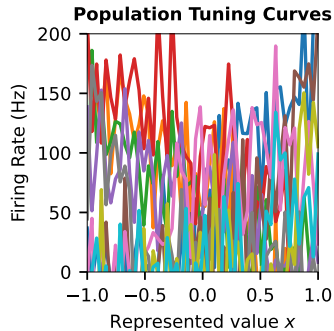
- ▶ **How to model?** Gaussian noise

NEF Principle 0: Noise

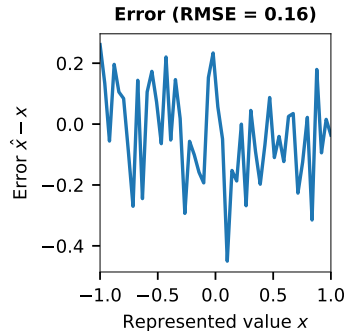
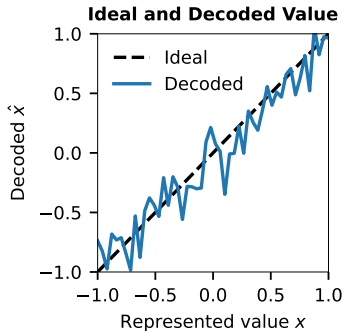
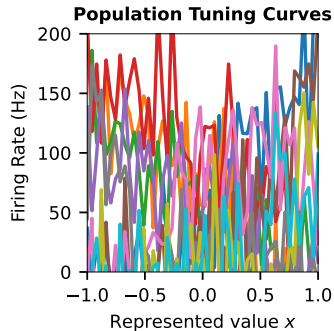
NEF Principle 0 – Noise

Biological neural systems are subject to significant amounts of noise from various sources. Any analysis of such systems must take the effects of noise into account.

Decoding Noisy $\mathbf{A}$ Without Taking Noise Into Account



Decoding Noisy Δ Accounting for Noise

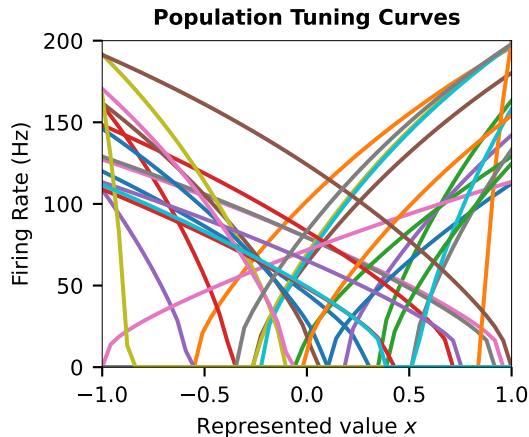


Summary: Building a model of neural representation (Encoding)

Encoding

- ▶ Select d , possible range $\mathbf{x} \in \mathbb{X}$, usually $\mathbb{X} = \{\mathbf{x} \mid \|\mathbf{x}\| \leq r, \mathbf{x} \in \mathbb{R}^d\}$ ($r = 1$)
- ▶ Select number of neurons n
- ▶ Select tuning curves, maximum rates
 $\Rightarrow \mathbf{e}_i, \alpha_i, J_i^{\text{bias}}$
 - ▶ Sample $\mathbf{e}_i$ from unit-sphere
 - ▶ Uniformly distribute x -intercept, maximum rate
- ▶ Encoding equation:

$$a_i(\mathbf{x}) = G[\alpha_i \langle \mathbf{e}_i, \mathbf{x} \rangle + J_i^{\text{bias}}]$$



Summary: Building a model of neural representation (Decoding)

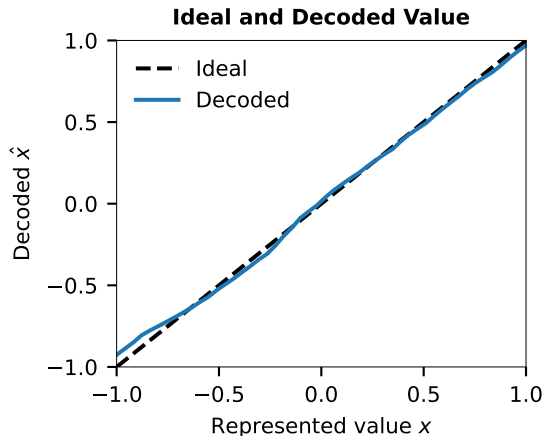
Decoding

- ▶ Uniformly sample N samples from $\mathbb{X}$, $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)$
- ▶ Compute $\mathbf{A}$, where $(\mathbf{A})_{ik} = a_i(\mathbf{x}_k)$
- ▶ Decoder computation:

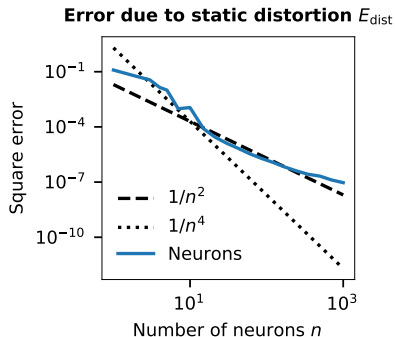
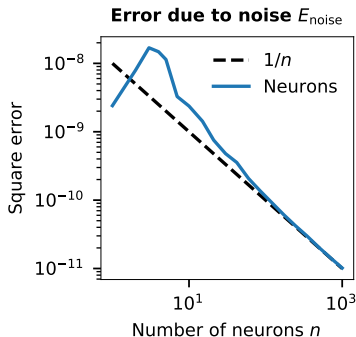
$$\mathbf{D}^T = (\mathbf{A}\mathbf{A}^T + N\sigma^2\mathbf{I})^{-1}\mathbf{A}\mathbf{X}^T$$

- ▶ Decoding equation:

$$\hat{\mathbf{X}} = \mathbf{D}\mathbf{A}$$

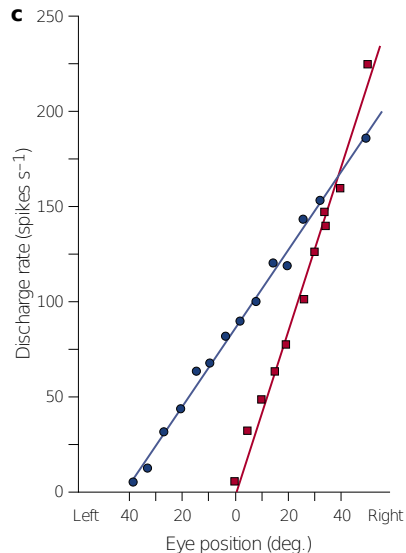
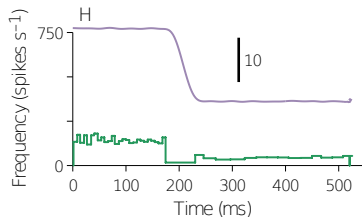
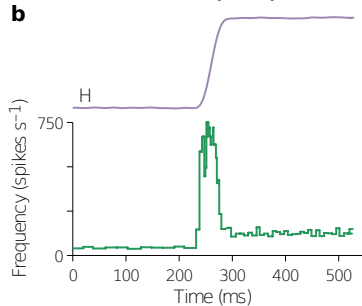
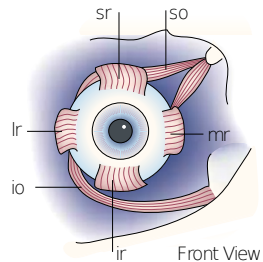
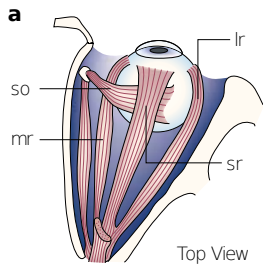


Analysing Sources of Errors



$$E = \underbrace{\frac{1}{2} \int_{-1}^1 \left(x - \sum_{i=1}^n d_i a_i(x) \right)^2 dx}_{E_{\text{dist}}} + \underbrace{\frac{1}{2} \sigma^2 \sum_{i=1}^n d_i^2}_{E_{\text{noise}}}$$

Example: Horizontal Eye Position (1D)



Example: Horizontal Eye Position (1D) (cont.)

► Step 1: System Description

- What is being represented?
 - x is the horizontal eye position
- What is the tuning curve shape?
 - Linear, low τ_{ref} , high τ_{RC}
 - $e_i \in \{1, -1\}$
 - Firing rates up to 300 s^{-1}

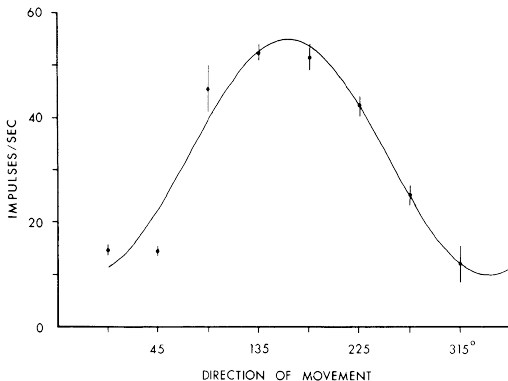
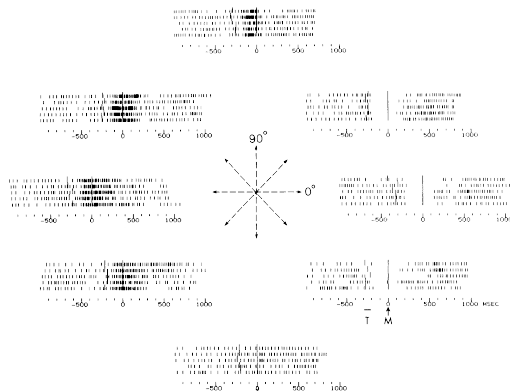
► Step 2: Design Specification

- Range of values
 - $\mathbb{X} = [-60, 60]$
- Amount of noise
 - About 20% of $\max(\mathbf{A})$

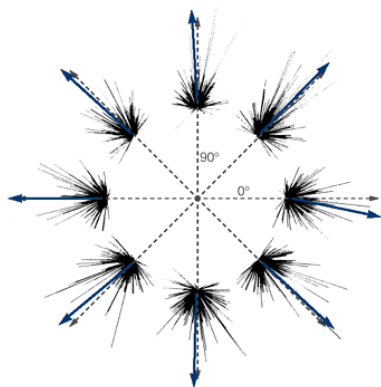
► Step 3: Implementation

- Choose tuning curve parameters
- Compute decoders

Example: Arm Movements (2D)



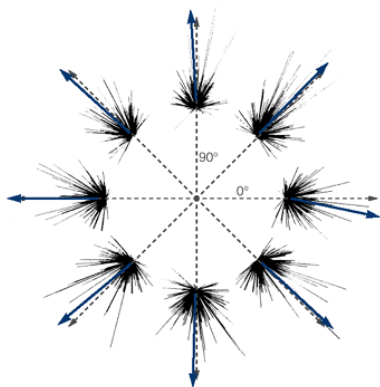
Example: Arm Movements (2D) (cont.)



- ▶ Experiment by Georgopoulos et al., 1982
- ▶ Preferred arm movement directions $\mathbf{e}_i$
- ▶ **Idea:** *Population Vectors*, decode using

$$\hat{\mathbf{x}} = \sum_{i=1}^n a_i(\mathbf{x}) \mathbf{e}_i = \mathbf{E} \mathbf{A}$$

Example: Arm Movements (2D) (cont.)

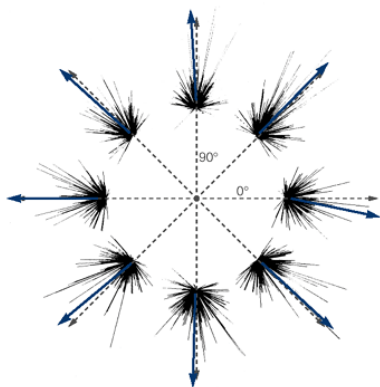


- ▶ Experiment by Georgopoulos et al., 1982
- ▶ Preferred arm movement directions $\mathbf{e}_i$
- ▶ **Idea:** *Population Vectors*, decode using

$$\hat{\mathbf{x}} = \sum_{i=1}^n a_i(\mathbf{x}) \mathbf{e}_i = \mathbf{E}\mathbf{A}$$

- ⊕ Good direction estimate

Example: Arm Movements (2D) (cont.)

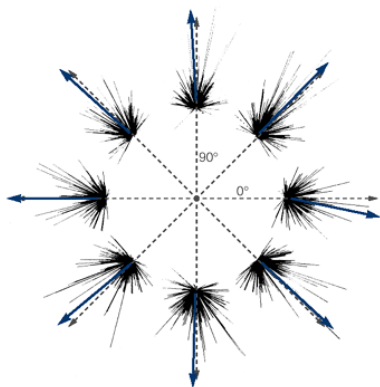


- ▶ Experiment by Georgopoulos et al., 1982
- ▶ Preferred arm movement directions $\mathbf{e}_i$
- ▶ **Idea:** *Population Vectors*, decode using

$$\hat{\mathbf{x}} = \sum_{i=1}^n a_i(\mathbf{x}) \mathbf{e}_i = \mathbf{E}\mathbf{A}$$

- ⊕ Good direction estimate
- ⊖ Cannot reconstruct magnitude

Example: Arm Movements (2D) (cont.)



- ▶ Experiment by Georgopoulos et al., 1982
- ▶ Preferred arm movement directions $\mathbf{e}_i$
- ▶ **Idea:** *Population Vectors*, decode using

$$\hat{\mathbf{x}} = \sum_{i=1}^n a_i(\mathbf{x}) \mathbf{e}_i = \mathbf{E}\mathbf{A}$$

- + Good direction estimate
- Cannot reconstruct magnitude

The NEF does not use population vectors!

Example: Arm Movements (2D) (cont.)

► Step 1: System Description

- What is being represented?
 - $\mathbf{x}$ the movement direction (or hand position)
- What is the tuning curve shape?
 - Bell-shaped
 - Encoders are randomly distributed along the unit circle
 - Firing rates up to 60 s^{-1}

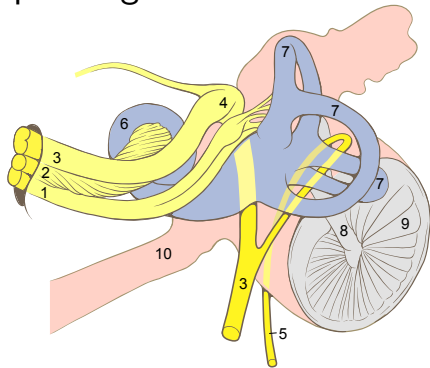
► Step 2: Design Specification

- Range of values
 - $\mathbb{X} = \{\mathbf{x} \mid \|\mathbf{x}\| \leq r, \mathbf{x} \in \mathbb{R}^2\}$
- Amount of noise
 - About 20% of $\max(\mathbf{A})$

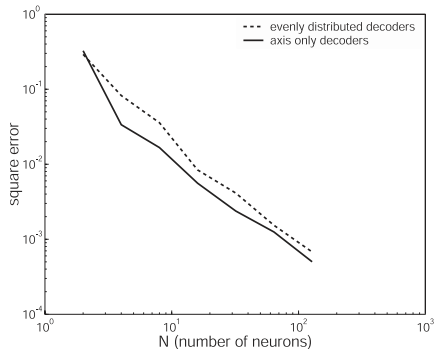
► Step 3: Implementation

- Choose tuning curve parameters
- Compute decoders

Example: Higher Dimensional Representation

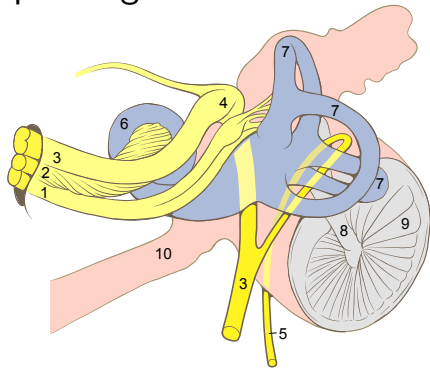


- ▶ Vestibular system senses head acceleration in 3D
- ▶ Axis aligned, must choose $\mathbf{e}_i \in \{[1, 0, 0], [-1, 0, 0], \dots, [0, 0, -1]\}$

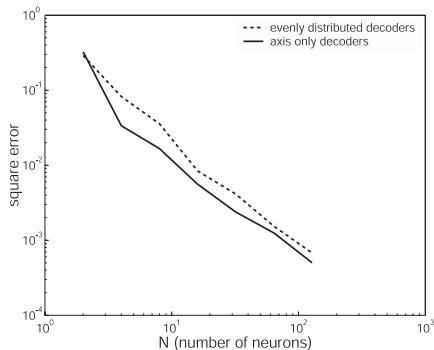


- ▶ Same as three 1D populations
- ▶ Slightly better precision

Example: Higher Dimensional Representation



- ▶ Vestibular system senses head acceleration in 3D
- ▶ Axis aligned, must choose $\mathbf{e}_i \in \{[1, 0, 0], [-1, 0, 0], \dots, [0, 0, -1]\}$



- ▶ Same as three 1D populations
- ▶ Slightly better precision
- ▶ **Encoders affect accuracy**

Administration

- ▶ **Assignment 1 has been released.**

Image sources

Title slide

“The Ultimate painting.”

Author: Clark Richert.

From Wikimedia.